arXiv:2305.05176v1 [cs.LG] 9 May 2023

FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance

Lingjiao Chen, Matei Zaharia, James Zou

Stanford University

Abstract

There is a rapidly growing number of large language models (LLMs) that users can query for a fee. We review the cost associated with querying popular LLM APIs—e.g. GPT-4, ChatGPT, J1-Jumbo—and find that these models have heterogeneous pricing structures, with fees that can differ by two orders of magnitude. In particular, using LLMs on large collections of queries and text can be expensive. Motivated by this, we outline and discuss three types of strategies that users can exploit to reduce the inference cost associated with using LLMs: 1) prompt adaptation, 2) LLM approximation, and 3) LLM cascade. As an example, we propose FrugalGPT, a simple yet flexible instantiation of LLM cascade which learns which combinations of LLMs to use for different queries in order to reduce cost and improve accuracy. Our experiments show that FrugalGPT can match the performance of the best individual LLM (e.g. GPT-4) with up to 98% cost reduction or improve the accuracy over GPT-4 by 4% with the same cost. The ideas and findings presented here lay a foundation for using LLMs sustainably and efficiently.

1 Introduction

We are in the midst of an explosion of large language models (LLMs). The alluring possibilities of using LLMs for large-scale applications such as commerce, science, and finance have led a growing number of companies (OpenAI, AI21, CoHere, etc.) to offer LLMs as services.

While LLMs such as GPT-4 achieves unprecedented performance in tasks such as question answering, using them for high-throughput applications can be very expensive. For example, ChatGPT is estimated to cost over $700,000 per day to operate [Cosa], and using GPT-4 to support customer service can cost a small business over $21,000 a month [Cosb]. In addition to the financial cost, using the largest LLMs encures substantial environmental and energy impact [BGMMS21, WRG+22], affecting the social welfare of current and future generations.

There are many LLMs now available via APIs and they charge heterogeneous prices. The cost of using a LLM API typically consists of three components: 1) prompt cost (proportional to the length of the prompt), 2) generation cost (proportional to the generation length), and 3) sometimes a fixed cost per query. We compared the cost associated with using 12 different commercial LLMs from mainstream providers including OpenAI, AI21, CoHere and Textsynth (Table 1). Their cost can differ by up to 2 orders of magnitudes: for example, the prompt cost for 10M tokens is $30 for OpenAI's GPT-4 but only $0.2 for GPT-J hosted by Textsyth.

Given the heterogeneous cost and quality, how to effectively and efficiently leverage the full set of LLM options is a key challenge for pracitioners. If the tasks are relatively simple, then aggregating multiple responses from GPT-J [WK21] (whose size is 30x smaller than GPT-3) offers performance similar to GPT-3 [ANC+22], leading to financial and environmental savings. However, the performance of GPT-J can be much worse on difficult tasks [TLI+23]. Moreover, relying on one API provider is not reliable if that provider becomes unavailable, potentially due to spiking demand. Existing model ensemble paradigms such as model cascade [VJ04, WLM11] and FrugalML [CZZ20, CZZ22] were designed for predictive tasks with a known set of labels and do not account for the full capabilities of LLM. How to use LLMs affordably and accurately therefore calls for new approaches.

Our contributions. In this paper, we lay out our vision of a flexible framework that uses LLM APIs to process natural language queries within a budget constraint, termed FrugalGPT. As shown in Figure

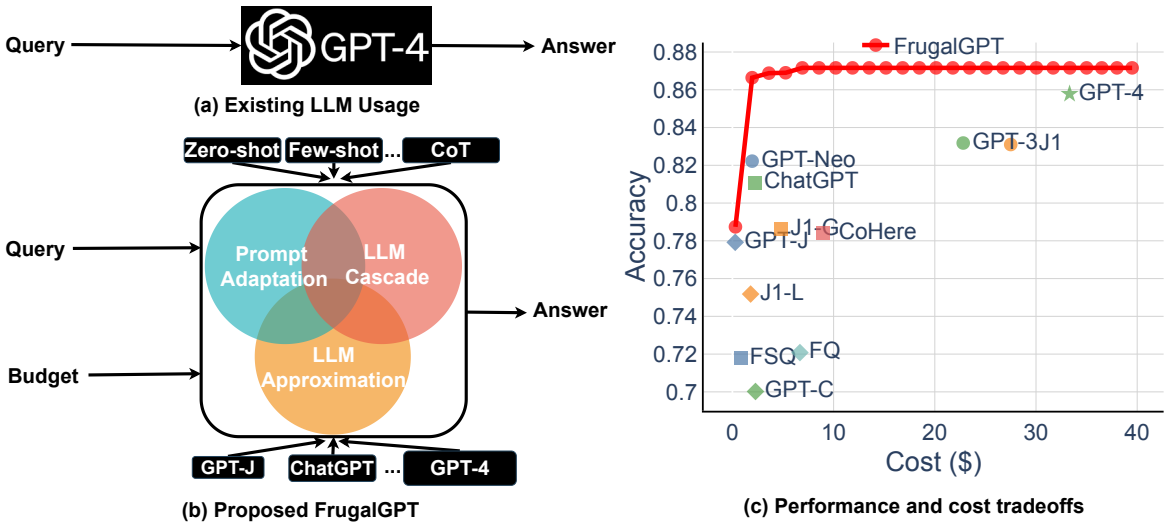


Figure 1: Our vision for reducing LLM cost while improving accuracy. (a) The standard usage sends queries to a single LLM (e.g. GPT-4), which can be expensive. (b) Our proposal is to use prompt adaption, LLM approximation and LLM cascade to reduce the inference cost. By optimizing over the selection of different LLM APIs (e.g., GPT-J, ChatGPT, and GPT-4) as well as prompting strategies (such as zero-shot [BMR+20], few-shot [LSZ+21], and chain-of-thought(CoT) [WWS+22]), we can achieve substantial efficiency gains. (c) On HEADLINES (a financial news dataset), FrugalGPT can reduce the inference cost by 98% while exceeding the performance of the best individual LLM (GPT-4).

1, we discuss three main strategies for cost reduction: *prompt adaptation*, *LLM approximation*, and *LLM cascade*. The prompt adaptation explores how to identify effective (often shorter) prompts to save cost. LLM approximation aims to create simpler and cheaper LLMs to match a powerful yet expensive LLM on specific tasks. LLM cascade focuses on how to adaptively choose which LLM APIs to use for different queries.

To illustrate the potential of these ideas, we implement and evaluate a simple version of FrugalGPT using LLM cascade. On each dataset and task, FrugalGPT learns how to adaptively triage different queries in the dataset to different combinations of LLMs, including ChatGPT [Cha], GPT-3 [BMR+20] and GPT-4 [Ope23]. Our experiments show that FrugalGPT can save up to 98% of the inference cost of the best individual LLM API while matching its performance on the downstream task. On the other hand, FrugalGPT can improve the performance by up to 4% with the same cost. We believe this is only the tip of the iceberg and we hope FrugalGPT opens a new window toward reducing LLMs' inference cost and improving its performances.

Related Works. Prompt Engineering. Prompt engineering has emerged as a discipline for crafting prompts to enhance LLMs' performance across various applications. Recent developments include few-shot [BMR+20], chain-of-thought [WWS+22], knowledge enhancement [LLL+21, KSL+22], and numerous other prompting techniques [MDL+23, KTF+22, ZSH+22, DGSG22]. Existing prompt engineering approaches often aim to provide more detailed task explanations and in-context examples, resulting in longer and more expensive prompts. In contrast, this paper explores the use of concise prompts to reduce costs.

Model Ensemble. Model ensembles, which involve combining multiple ML models for prediction, have gained popularity in supervised learning [VJ04, Fri02], unsupervised learning [YLLL14], semi-supervised learning [GDMR22], and weakly supervised learning [DSP+17]. Model ensembles typically require white-box access to multiple models for training purposes, but LLM APIs are often black-box. Moreover, model ensembles necessitate querying all models for a single query, thereby increasing costs.

System Optimization for LLMs. Numerous efforts have aimed to accelerate the training and inference time of modern deep learning models through system optimization [HMD15, CHSV17, Cas19, JZA19, RRWN11]. Recent work focuses on post-training quantization [BHS+22, YLW+23, XLS+22], training pipeline parallelism [LZG+21], and hardware-aware pruning [KFA23] tailored for LLMs. System optimization requires modifications to LLMs' internal states (e.g., model weights), but many commercial

LLM APIs do not release their models. Additionally, the rapidly increasing size of LLMs renders retraining highly expensive.

ML-as-a-Service. LLM APIs constitute a crucial component of the rapidly expanding machine-learning-as-a-service (MLaaS) industry. Recent studies have demonstrated the diversity of different ML APIs' predictions [BG18, KNL+20, CCZZ21] and proposed strategies for leveraging various classification ML APIs to improve performance [CZZ20, CZZ22]. The outputs of LLM APIs encompass the entire natural language space, but existing work requires a fixed (and known) label set. Moreover, both prompt choices and LLM API selections significantly impact generative tasks' performance, resulting in a considerably larger optimization space than standard classification.

The remaining part of the paper is organized as follows. We start by offering more context and the problem statement in Section 2. Next in Section 3, we present our visions on how to use LLM APIs affordability and accurately. Section 4 shows the empirical benefits of FrugalGPT using real-world LLM APIs (including GPT-3, ChatGPT, and GPT-4). Finally, we discuss future prospects in Section 5.

2 Scope and Problem Statement

Natural language query answering. In this paper, we concentrate on the standard natural language query answering task, where the objective is to answer a query q sampled from a natural language query distribution $\mathcal{Q}$. Various real-world natural language tasks, such as news classification, reading comprehension, and commonsense reasoning, can be formulated as query-answering problems.

LLM marketplace. We consider answering queries via the LLM market, which comprises K different LLM APIs, denoted by $\{f_i(\cdot)\}_{i=1}^K$. Each $f_i(\cdot) : \mathcal{P} \mapsto \mathcal{A}$ is a function that, given a prompt p from the prompt space $\mathcal{P}$, generates an answer from the answer distribution $\mathcal{A}$. Note that to use LLM APIs, one has to convert each query q to some corresponding prompt first. LLM APIs are associated with their own *cost*, typically consisting of three components: a portion proportional to the length of the prompt, a portion proportional to the length of the generated answer, and (sometimes) a fixed cost per query. Formally, given a prompt p, the cost of using the ith LLM API is denoted by $c_i(p) \triangleq \tilde{c}_{i,2}\|f_i(p)\| + \tilde{c}_{i,1}\|p\| + \tilde{c}_{i,0}$, where $\tilde{c}_{i,j}, j = 0, 1, 2$ are constants.

An illustrative example. Adapting the case study provided by [Cosa], assume a small business operates a customer service using GPT-4. The company caters to 15,000 customers each month, with each customer asking three questions twice a week, totaling 360,000 queries per month. Suppose for each question, its corresponding prompt averages 1800 tokens, and the answer is around 80 tokens. Considering that the input and response costs of GPT-4 are \$0.03 and \$0.06 per thousand tokens, respectively, the total monthly cost amounts to $360 \times (\$0.03 \times 1800 + \$0.06 \times 80) \approx \$21.2K$. Such a high cost is prohibitive for many small businesses.

Problem statement: budget-aware LLM API usage. Our primary goal in this paper is *leveraging LLM APIs within a budget constraint*. Formally, this can be formulated as maximizing the overall task performance $\mathbb{E}_{(q,a)\in\mathcal{Q}\times\mathcal{A}}[r(a, \hat{a}(s, q))]$, while ensuring the average cost is bounded by a user-defined value b, i.e., $\mathbb{E}_{(q,a)\in\mathcal{Q}\times\mathcal{A}}[c(s, q)] \leq b$. Here, a denotes the correct answer to the query q, $\hat{a}(s, q)$ is the generated answer by some strategy s for query q, and $c(s, q)$ is the associated cost for processing query q using strategy s. The reward function $r(\cdot, \cdot)$ measures how closely the generated answer aligns with the correct one. It is crucial to note that the search space for the strategy is vast, encompassing factors such as which prompts to use, which LLM APIs to employ, and how to aggregate their responses.

3 How to Use LLMs Affordably and Accurately

Now we present our vision on how to use LLM APIs within a budget. As shown in Figure 1 (b), we discuss three cost-reduction strategies, namely, prompt adaptation, LLM approximation, and LLM cascade.

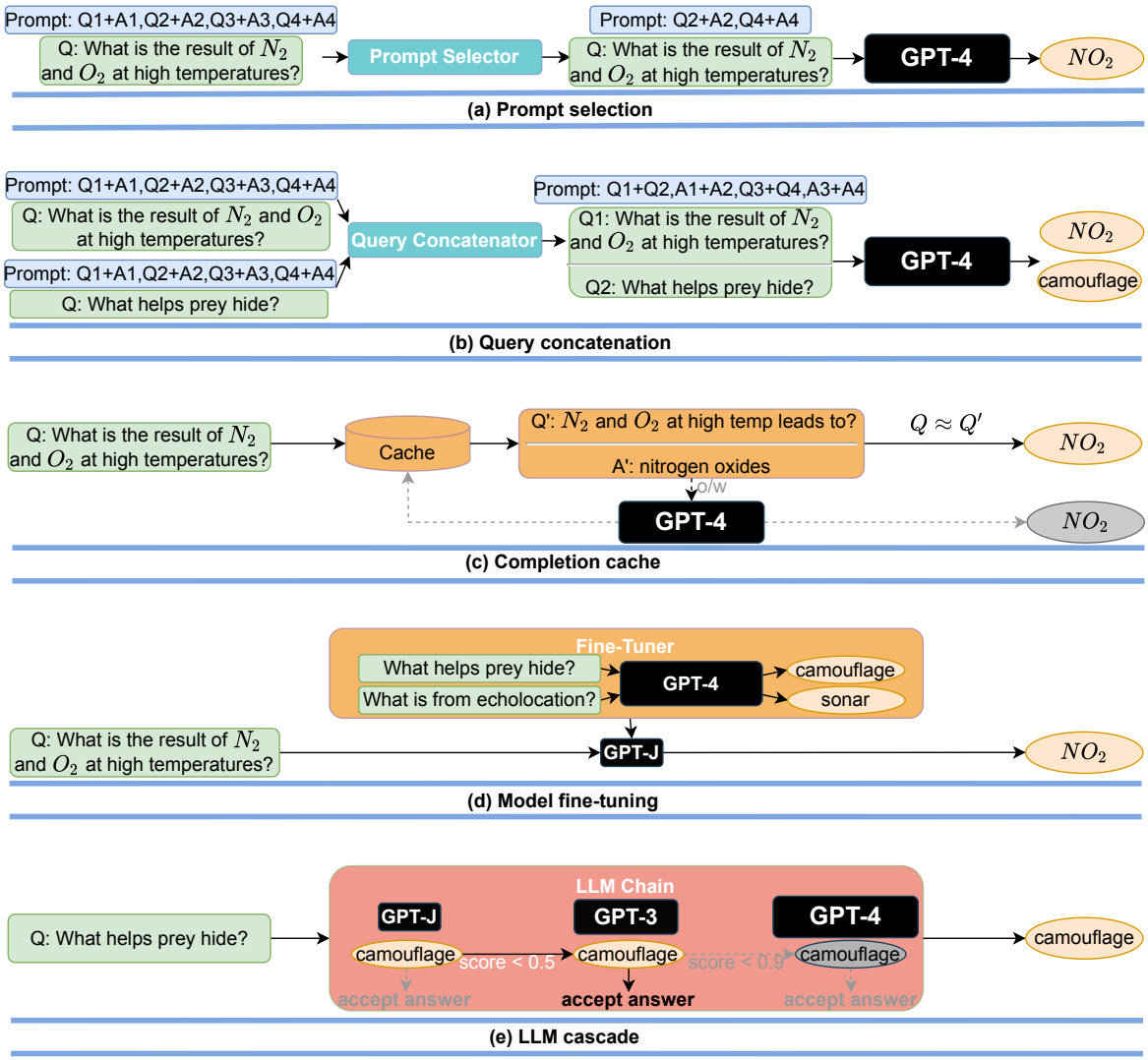


Figure 2: Illustrations of cost-saving strategies. (a) Prompt selection uses a subset of in-context examples as the prompt to reduce the size of the prompt. (b) Query concatenation aggregates multiple queries to share prompts. (c) Completion cache stores and reuses an LLM API's response when a similar query is asked. (d) Model fine-tuning uses expensive LLMs' responses to fine-tune cheap LLMs. (e) LLM cascade employs different LLM APIs for different queries.

Strategy 1: Prompt adaptation. The cost of an LLM query increases linearly with the size of the prompt. Consequently, a logical approach to reduce the cost of using LLM APIs involves decreasing the prompt's size, a process we refer to as prompt adaptation. *Prompt selection* (as illustrated in Figure 2 (a)) is a natural example of prompt adaptation: rather than employing a prompt containing numerous examples that demonstrate how to perform a task, one can retain a small subset of examples in the prompt. This results in a smaller prompt and subsequently lower cost. An intriguing challenge of prompt selection lies in determining which examples to maintain for various queries without compromising task performance.

An additional instantiation is *query concatenation* (Figure 2 (b)). It is important to note that processing queries individually necessitates sending the same prompt to an LLM API multiple times. Therefore, the fundamental concept of query concatenation involves sending the prompt only once to the LLM API while allowing it to address multiple queries, thereby preventing redundant prompt processing. To accomplish this, several queries must be concatenated into a single query, and the prompt must explicitly request the LLM API to process multiple queries. For instance, to handle two queries using one prompt, the examples presented in the prompt can include both queries followed by their corresponding answers.

Strategy 2: LLM approximation. The concept of *LLM approximation* is quite simple: if an LLM API is too costly to utilize, one can approximate it using more affordable models or infrastructures. One example is the *completion cache*: as depicted in Figure 2 (c), the fundamental idea involves storing the response locally in a cache (e.g., a database) when submitting a query to an LLM API. To process a new query, we first verify if a similar query has been previously answered. If so, the response is retrieved from the cache. An LLM API is invoked only if no similar query is discovered in the cache. The completion cache provides substantial cost savings when similar queries are frequently posed. For instance, consider a search engine powered by an LLM API. If numerous users search for the same or similar keywords simultaneously, the completion cache facilitates answering all their queries by invoking the LLM only once.

Another example of LLM approximation is *model fine-tuning*. As shown in Figure 2(d), this process consists of three steps: first, collect a powerful but expensive LLM API's responses to a few queries; second, use the responses to fine-tune a smaller and more affordable AI model; and finally, employ the fine-tuned model for new queries. In addition to cost savings, the fine-tuned model often does not require lengthy prompts, thus providing latency improvements as a byproduct.

Strategy 3: LLM cascade. The increasing availability of LLM APIs with heterogeneous performance and costs presents a unique opportunity for data-adaptive LLM selection. Different LLM APIs have their own strengths and weaknesses for various queries. Consequently, appropriately selecting which LLMs to use can provide both cost reduction and performance improvements. *LLM cascade*, as illustrated in Figure 2 (e), is one such example. *LLM cascade* sends a query to a list of LLM APIs sequentially. If one LLM API's response is reliable, then its response is returned, and no further LLMs in the list are needed. The remaining LLM APIs are queried only if the previous APIs' generations are deemed insufficiently reliable. Query cost is significantly reduced if the first few APIs are relatively inexpensive and produce reliable generations.

The key components of LLM cascade consist of two elements: (i) a generation scoring function and (ii) an LLM router. The generation scoring function, denoted by $g(\cdot,\cdot) : \mathcal{Q} \times \mathcal{A} \mapsto [0,1]$, generates a reliability score given a query and an answer produced by an LLM API. The LLM router selects m LLM APIs to include in the list. Let $\boldsymbol{L} \in [K]^m$ denote the indexes of the m APIs selected by the router. Given a new query, it iteratively invokes the ith API in the list to obtain an answer $f_{L_i}(q)$. Then, it uses the scoring function to generate a score $g(q, f_{L_i}(q))$. It returns the generation if the score is higher than a threshold $\boldsymbol{\tau}_i$, and queries the next service otherwise.

The scoring function can be obtained by training a simple regression model that learns whether a generation is correct from the query and a generated answer. Learning the selected list $\boldsymbol{L}$ and the

threshold vectors $\boldsymbol{\tau}$ can be modeled as a constraint optimization problem:

$$\max_{\boldsymbol{L},\boldsymbol{\tau}} \ \mathbb{E}\left[r(a, f_{L_z}(q))\right]$$
$$s.t. \ \mathbb{E}\left[\sum_{i=1}^{z} \tilde{c}_{L_i,2}\|f_{L_i}(q)\| + \tilde{c}_{L_i,1}\|q\| + \tilde{c}_{L_i,0}\right] \leq b,$$
$$z = \arg\min_i g(q, f_{L_i}(q)) \geq \boldsymbol{\tau}_i$$

Here, z denotes the LLM API at which the router stops and returns the answer, the first constraint ensures the average cost is bounded by the budget, and the objective measures the quality of the generation $f_{L_z}(q)$ for a query q compared to the true answer a. This problem is inherently a mixed-integer optimization and thus computationally expensive to solve. To address this issue, we develop a specialized optimizer that (i) prunes the search space of $\boldsymbol{L}$ by ignoring any list of LLMs with small answer disagreement, and (ii) approximates the objective by interpolating it within a few samples. This results in an efficient implementation with satisfactory performance, as shown later in Figure 5.

Compositions. Combining approaches within and across different strategies can lead to further cost reduction and performance enhancement. For instance, *joint prompt and LLM selection* is a composition of prompt selection and LLM cascade: for a given query, it searches for the smallest prompt and most affordable LLM that achieves satisfactory task performance. Another example is to search across both existing LLM APIs and fine-tuned models. It is important to note that the composition of different approaches also increases the computational costs for training. Consequently, this paves the way for investigating trade-offs between query costs, task performance, and computational costs.

4 LLM Cascade Reduces Cost and Improves Accuracy

In this section, we present an empirical study on the FrugalGPT LLM cascade. Our goals are three-fold: (i) understand what a simple instantiation of LLM cascade learns, (ii) quantify the cost savings attained by FrugalGPT while matching the best individual LLM API's performance, and (iii) measure the trade-offs between performance and cost enabled by FrugalGPT.

Setups: LLM APIs, Tasks, Datasets, and FrugalGPT instances. We have selected 12 LLM APIs from 5 mainstream providers, namely, OpenAI [Ope], AI21 [AI2], CoHere [CoH], Textsynth [Tex], and ForeFrontAI [FFA]. The details are summarized in Table 1. FrugalGPT has been developed on top of these APIs and evaluated on a range of datasets belonging to different tasks, including HEADLINES [SK21], OVERRULING [ZGA+21], and COQA [RCM19]. The summary of these datasets is presented in Table 2. HEADLINES is a financial news dataset whose goal is to determine the gold price trend (up, down, neutral, or none) by reading financial news titles. This is especially useful for filtering relevant news in financial markets. OVERRULING is a legal document dataset where the goal is to determine whether a given sentence is an overruling, i.e., rejecting previous legal cases. COQA is a reading comprehension dataset developed in a conversational setting, which we have adapted as a direct query answering task. We focus on the LLM cascade approach with a cascade length of 3, as this simplifies the optimization space and already demonstrates good results. Each dataset is randomly split into a training set to learn the LLM cascade and a test set for evaluation.

A Case Study. Let us begin with a case study on the HEADLINES dataset. We set the budget to be \$6.5, which is one-fifth of GPT-4's cost. We employ a DistilBERT [SDCW19] tailored to regression as the scoring function. It is important to note that DistilBERT is considerably smaller and therefore less expensive than all LLMs considered here. As depicted in Figure 3 (a), the learned FrugalGPT sequentially calls GPT-J, J1-L, and GPT-4. For any given query, it first extracts an answer from GPT-J. If the score of this answer is greater than 0.96, the answer is accepted as the final response. Otherwise, J1-L is queried. J1-L's answer is accepted as the final response if its score is greater than 0.37; otherwise, GPT-4 is invoked to obtain the final answer. Interestingly, this approach outperforms GPT-4 for numerous queries. For instance, given a headline "Gold off the lows after dismal U.S. GDP data" from NASDAQ, FrugalGPT accurately predicts that the price is going down, while GPT-4

Table 1: Summary of commercial LLM APIs. We use 12 LLM APIs from 5 providers. The cost was retrieved in March 2023. The cost can have three additive components: input (proportional to the number of input tokens), output (proportional to the number of generated tokens) and a fixed cost per request. The LLMs's costs can differ by up to 2 orders of magnitudes. For example, to process 10M input tokens, GPT-J from Textsynth costs only \$0.2, but OpenAI's GPT-4 needs \$30.

Provider	**API**	**Size/B**	**Cost (USD)**		
			10M input tokens	**10M output tokens**	**request**
OpenAI	GPT-Curie	6.7	2	2	0
	ChatGPT	NA	2	2	0
	GPT-3	175	20	20	0
	GPT-4	NA	30	60	0
AI21	J1-Large	7.5	0	30	0.0003
	J1-Grande	17	0	80	0.0008
	J1-Jumbo	178	0	250	0.005
Cohere	Xlarge	52	10	10	0
ForeFrontAI	QA	16	5.8	5.8	0
Textsynth	GPT-J	6	0.2	5	0
	FAIRSEQ	13	0.6	15	0
	GPT-Neox	20	1.4	35	0

Table 2: Summary of datasets used in the FrugalGPT LLM cascade experiments.

Dataset	Domain	Size	#Examples in the prompt
HEADLINES	Finance	10000	8
OVERRULING	Law	2400	5
COQA	Passage Reading	7982	2

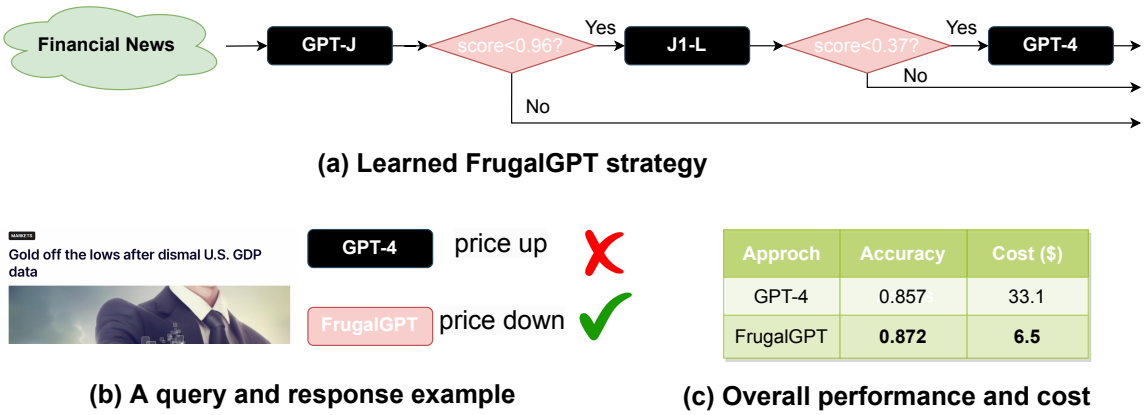
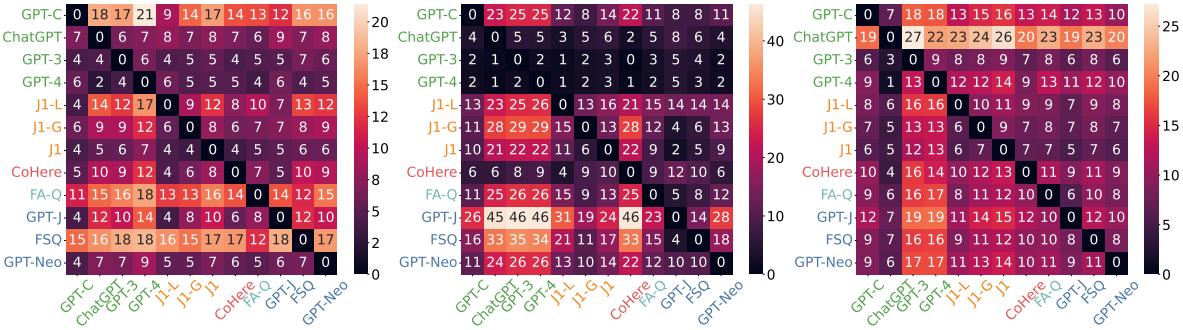


Figure 3: A case study of FrugalGPT on the HEADLINES dataset. (a) The cascade strategy that FrugalGPT learned on this dataset with overall budget \$6.5, one fifth of GPT-4's cost. FrugalGPT avoids querying GPT-4 as long as GPT-J and J1-L produce high-quality answers. (b) Sometimes GPT-4 makes a mistake, but FrugalGPT learns to use the correct answers by J-1 and GPT-J. (c) Overall, we observe that FrugalGPT reduces the cost by 80%, while improves the accuracy by 1.5% compared to GPT-4.



(a) HEADLINES (b) OVERRULING (c) COQA

Figure 4: Maximum performance improvement (MPI) of each pair of LLMs. (a), (b), and (c) correspond to the three datasets, separately. One entry indicates the percent of cases that the LLM on its row is wrong but the LLM on its column gives the right answer. Overall, we observe that cheap LLMs can be complementary to the expensive ones quite often. For example, for about 6% of the data, GPT-4 makes a mistake but GPJ-J (or J-L or GPT-C) gives the right answer on HEADLINES.

provides an incorrect answer (as shown in Figure 3(b)). Overall, FrugalGPT results in both accuracy gains and cost reduction. As illustrated in Figure 3(c), its cost is reduced by 80%, while the accuracy is even 1.5% higher.

LLM diversity. Why can multiple LLM APIs potentially produce better performance than the best individual LLM? In essence, this is due to generation diversity: even an inexpensive LLM can sometimes correctly answer queries on which a more expensive LLM fails. To measure this diversity, we use the maximum performance improvement, or *MPI*. The MPI of LLM A with respect to LLM B is the probability that LLM A generates the correct answer while LLM B provides incorrect ones. This metric essentially measures the maximum performance gains achievable by invoking LLM A in addition to LLM B.

MPI between each pair of LLM APIs for all datasets is displayed in Figure 4. Overall, we observe significant potential within the LLM marketplace. For instance, GPT-C, GPT-J, and J1-L can all enhance GPT-4's performance by up to 6% on the HEADLINES dataset. On the COQA dataset, there are 13% of data points where GPT-4 makes an error, but GPT-3 provides the correct answer. Although these improvement upper bounds may not always be attainable, they do demonstrate the possibility of utilizing more affordable services to achieve better performance.

Table 3: Cost savings by FrugalGPT to match the best individual LLM's performance.

Dataset	Best invidual LLM	Cost to reach the same accuracy		Cost Savings
		Best individual LLM	FrugalGPT	
HEADLINES	GPT-4	33.1	0.6	98.3%
OVERULLING	GPT-4	9.7	2.6	73.3%
COQA	GPT-3	72.5	29.6	59.2%

Cost Savings. Subsequently, we examine whether FrugalGPT can reduce costs while maintaining accuracy and, if so, by how much. Table 3 displays the overall cost savings of FrugalGPT, which range from 50% to 98%. This is feasible because FrugalGPT identifies the queries that can be accurately answered by smaller LLMs and, as a result, only invokes those cost-effective LLMs. Powerful but expensive LLMs, such as GPT-4, are utilized only for challenging queries detected by FrugalGPT.

Performance and Cost Trade-offs. Now, we investigate the trade-offs between performance and cost achieved by FrugalGPT, as illustrated in Figure 5. Several interesting observations can be made.

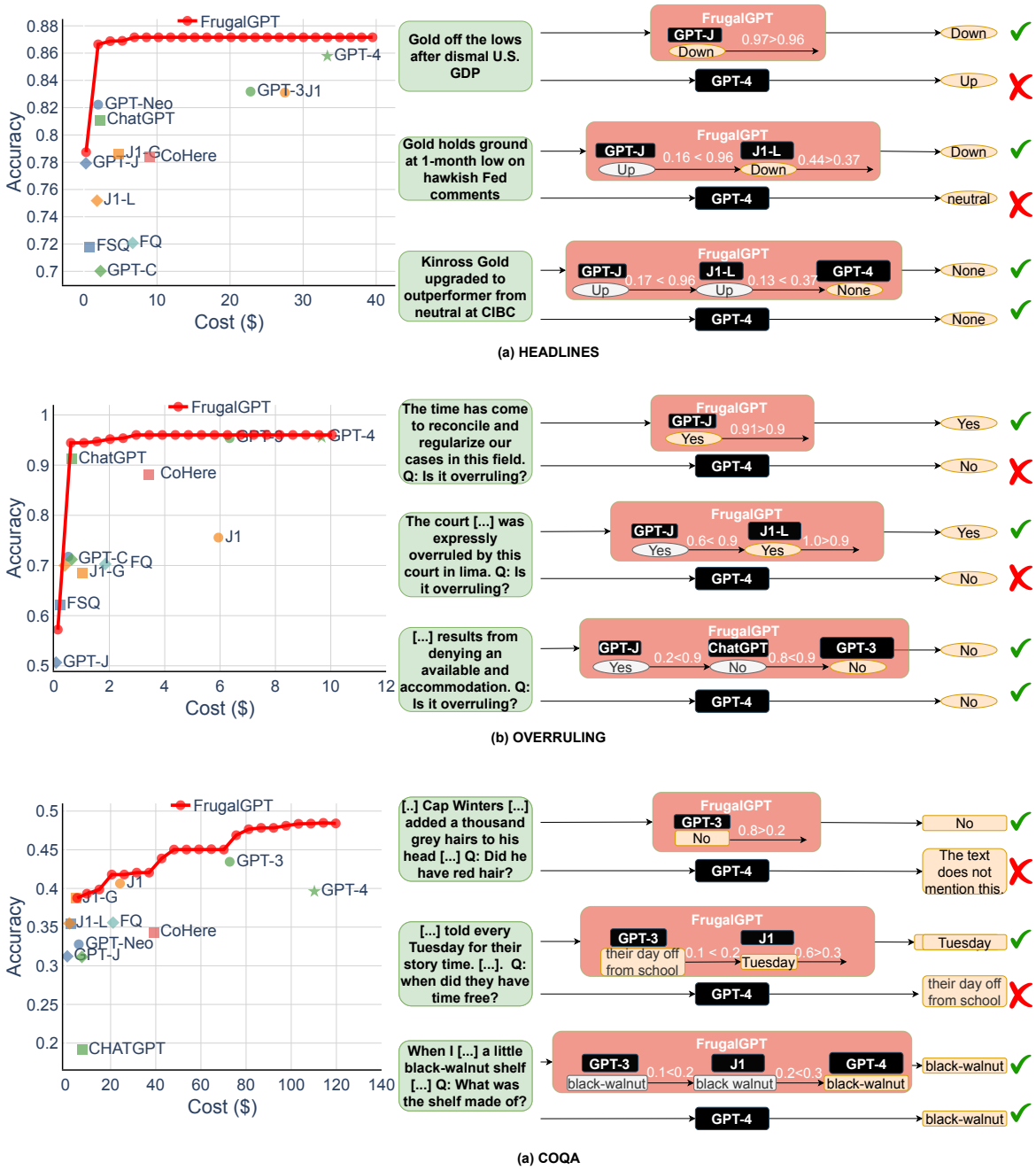


Figure 5: Accuracy and cost tradeoffs achieved by FrugalGPT. Overall, FrugalGPT often achieves the same performance of the best individual LLM API (e.g., GPT-4) with orders of magnitudes smaller cost. When incurring the same cost, FrugalGPT can improves the accuracy by up to 5%. Examples of LLM cascade for each dataset are shown on the right.

First, the cost ranking of different LLM APIs is not fixed. For instance, J1 is the second most expensive LLM on the HEADLINES dataset, while GPT-3 holds that position on the OVERRULING and COQA datasets. This is primarily due to the heterogeneous pricing mechanism: J1 incurs a high cost for each generated token but charges nothing for input tokens, whereas GPT-3 charges for both input and output tokens. Moreover, more expensive LLM APIs sometimes result in worse performance than their cheaper counterparts. For example, J1 is costlier than GPT-3 on HEADLINES, but its performance is inferior. These observations underscore the importance of aptly selecting LLM APIs, even in the absence of budget constraints.

Next, we note that FrugalGPT enables smooth performance-cost trade-offs across all evaluated datasets. This offers flexible choices to LLM users and potentially helps LLM API providers save energy and reduce carbon emissions. In fact, FrugalGPT can simultaneously reduce costs and improve accuracy. For example, on the OVERRULING dataset, FrugalGPT achieves a 1% accuracy gain while reducing costs by 73% compared to the best LLM API, GPT-4. This is likely because FrugalGPT integrates knowledge from multiple LLMs.

The example queries shown in Figure 5 further aid in understanding why FrugalGPT can simultaneously improve performance and reduce costs. GPT-4 makes mistakes on some queries (e.g., the first example in part (a)), but some low-cost APIs provide correct predictions. FrugalGPT accurately identifies those queries and relies solely on the inexpensive APIs. For example, GPT-4 incorrectly infers no overruling from the legal statement "The time has come to reconcile and regularize our cases in this field," as shown in Figure 5(b). However, FrugalGPT accepts GPT-J's correct answer, avoiding the use of expensive LLMs and improving overall performance. Naturally, a single LLM API is not always correct; LLM cascade overcomes this by employing a chain of LLM APIs. For example, in the second example shown in Figure 5(a), FrugalGPT identifies that GPT-J's generation may not be reliable and turns to the second LLM in the chain, J1-L, to find the correct answer. Again, GPT-4 provides the wrong answer. FrugalGPT is not perfect, and there remains ample room for cost reduction. For example, in the third example in Figure 5(c), all LLM APIs in the chain give the same answer. However, FrugalGPT is unsure if the first LLMs are correct, resulting in the need to query all LLMs in the chain. Identifying how to avoid such cases remains an open problem.

5 Discussions, Limitations and Future Prospects

The substantial cost of employing LLMs in real-world scenarios presents a considerable barrier to their widespread usage. In this paper, we outline and discuss practical strategies for reducing the inference cost of using LLM APIs. We also developed FrugalGPT to illustrate one of the cost-saving strategies, LLM cascade. Our empirical findings show that FrugalGPT can reduce costs by up to 98% while preserving the performance of cutting-edge LLMs.

FrugalGPT lays the groundwork for optimizing task performance with LLM APIs under budget constraints; however, it has some limitations. To train the LLM cascade strategy in FrugalGPT, we need some labeled examples. And in order for the cascade to work well, the training examples should be from the same or similar distribution as the test examples. Moreover, learning the LLM cascade itself requires resources. We view this as an one-time upfront cost; this is beneficial when the final query dataset is larger than the data used to train the cascade. There are also other promising strategies for cost saving, such as speeding up attention computation itself, that we do not discuss here. Given the rapid development of LLM, this paper is not meant to be comprehensive or to provide a definitive solution. Our goal is to lay a foundation for this important research agenda and to demonstrate that even simple cascade can already achieve promising savings.

There are also many related directions for future exploration. While FrugalGPT concentrates on balancing performance and cost, real-world applications call for the evaluation of other critical factors, including latency, fairness, privacy, and environmental impact. Incorporating these elements into optimization methodologies while maintaining performance and cost-effectiveness is an important avenue for future research. Furthermore, utilizing LLMs in risk-critical applications necessitates the careful quantification of uncertainty in LLM-generated outputs. As the field progresses, addressing the environmental ramifications of training and deploying LLMs demands a joint effort from LLM users and API providers. The continuous evolution of LLMs and their applications will inevitably unveil new challenges and opportunities, fostering further research and development in this dynamic field.

References

[AI2] AI21 LLM API. https://www.ai21.com/. Accessed: 2023-03-31.

[ANC+22] Simran Arora, Avanika Narayan, Mayee F Chen, Laurel J Orr, Neel Guha, Kush Bhatia, Ines Chami, Frederic Sala, and Christopher Ré. Ask me anything: A simple strategy for prompting language models. *arXiv preprint arXiv:2210.02441*, 2022.

[BG18] Joy Buolamwini and Timnit Gebru. Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Conference on fairness, accountability and transparency*, pages 77–91. PMLR, 2018.

[BGMMS21] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, 2021.

[BHS+22] Haoli Bai, Lu Hou, Lifeng Shang, Xin Jiang, Irwin King, and Michael R Lyu. Towards efficient post-training quantization of pre-trained language models. *Advances in Neural Information Processing Systems*, 35:1405–1418, 2022.

[BMR+20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[Cas19] Stephen Cass. Taking ai to the edge: Google's tpu now comes in a maker-friendly package. *IEEE Spectrum*, 56(5):16–17, 2019.

[CCZZ21] Lingjiao Chen, Tracy Cai, Matei Zaharia, and James Zou. Did the model change? efficiently assessing machine learning api shifts. *arXiv preprint arXiv:2107.14203*, 2021.

[Cha] ChatGPT Announcement. https://openai.com/blog/chatgpt. Accessed: 2023-03-31.

[CHSV17] Zhaowei Cai, Xiaodong He, Jian Sun, and Nuno Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5918–5926, 2017.

[CoH] CoHere LLM API. https://cohere.com/. Accessed: 2023-03-31.

[Cosa] Cost estimation of using GPT-3 for real applications. https://www.semianalysis.com/p/the-inference-cost-of-search-disruption. Accessed: 2023-03-31.

[Cosb] Cost estimation of using GPT-3 for real applications. https://neoteric.eu/blog/how-much-does-it-cost-to-use-gpt-models-gpt-3-pricing-explained. Accessed: 2023-03-31.

[CZZ20] Lingjiao Chen, Matei Zaharia, and James Y Zou. Frugalml: How to use ml prediction apis more accurately and cheaply. *Advances in neural information processing systems*, 33:10685–10696, 2020.

[CZZ22] Lingjiao Chen, Matei Zaharia, and James Zou. Efficient online ml api selection for multi-label classification tasks. In *International Conference on Machine Learning*, pages 3716–3746. PMLR, 2022.

[DGSG22] Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. Successive prompting for decomposing complex questions. *arXiv preprint arXiv:2212.04092*, 2022.

[DSP+17] Ali Diba, Vivek Sharma, Ali Pazandeh, Hamed Pirsiavash, and Luc Van Gool. Weakly supervised cascaded convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 914–922, 2017.

[FFA] forefront AI LLM API. https://beta.forefront.ai/. Accessed: 2023-03-31.

[Fri02] Jerome H Friedman. Stochastic gradient boosting. *Computational statistics & data analysis*, 38(4):367–378, 2002.

[GDMR22] Ashit Gupta, Anirudh Deodhar, Tathagata Mukherjee, and Venkataramana Runkana. Semi-supervised cascaded clustering for classification of noisy label data. *arXiv preprint arXiv:2205.02209*, 2022.

[HMD15] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.

[JZA19] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems*, 1:1–13, 2019.

[KFA23] Eldar Kurtic, Elias Frantar, and Dan Alistarh. Ziplm: Hardware-aware structured pruning of language models. *arXiv preprint arXiv:2302.04089*, 2023.

[KNL+20] Allison Koenecke, Andrew Nam, Emily Lake, Joe Nudell, Minnie Quartey, Zion Mengesha, Connor Toups, John R Rickford, Dan Jurafsky, and Sharad Goel. Racial disparities in automated speech recognition. *Proceedings of the National Academy of Sciences*, 117(14):7684–7689, 2020.

[KSL+22] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp. *arXiv preprint arXiv:2212.14024*, 2022.

[KTF+22] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.

[LLL+21] Jiacheng Liu, Alisa Liu, Ximing Lu, Sean Welleck, Peter West, Ronan Le Bras, Yejin Choi, and Hannaneh Hajishirzi. Generated knowledge prompting for commonsense reasoning. *arXiv preprint arXiv:2110.08387*, 2021.

[LSZ+21] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? *arXiv preprint arXiv:2101.06804*, 2021.

[LZG+21] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. Terapipe: Token-level pipeline parallelism for training large-scale language models. In *International Conference on Machine Learning*, pages 6543–6552. PMLR, 2021.

[MDL+23] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Augmented language models: a survey. *arXiv preprint arXiv:2302.07842*, 2023.

[Ope] OpenAI LLM API. https://platform.openai.com/. Accessed: 2023-03-31.

[Ope23] OpenAI. Gpt-4 technical report. *arXiv preprint https://arxiv.org/abs/2303.08774*, 2023.

[RCM19] Siva Reddy, Danqi Chen, and Christopher D Manning. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019.

[RRWN11] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in neural information processing systems*, 24, 2011.

[SDCW19] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[SK21] Ankur Sinha and Tanmay Khandait. Impact of news on the commodity market: Dataset and results. In *Advances in Information and Communication: Proceedings of the 2021 Future of Information and Communication Conference (FICC), Volume 2*, pages 589–601. Springer, 2021.

[Tex] Textsynth LLM API. https://textsynth.com/. Accessed: 2023-03-31.

[TLI+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[VJ04] Paul Viola and Michael J Jones. Robust real-time face detection. *International journal of computer vision*, 57:137–154, 2004.

[WK21] Ben Wang and Aran Komatsuzaki. Gpt-j-6b: A 6 billion parameter autoregressive language model, 2021.

[WLM11] Lidan Wang, Jimmy Lin, and Donald Metzler. A cascade ranking model for efficient ranked retrieval. In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*, pages 105–114, 2011.

[WRG+22] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems*, 4:795–813, 2022.

[WWS+22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

[XLS+22] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

[YLLL14] Fan Yang, Xuan Li, Qianmu Li, and Tao Li. Exploring the diversity in cluster ensemble generation: Random sampling and random projection. *Expert Systems with Applications*, 41(10):4844–4866, 2014.

[YLW+23] Zhewei Yao, Cheng Li, Xiaoxia Wu, Stephen Youn, and Yuxiong He. A comprehensive study on post-training quantization for large language models. *arXiv preprint arXiv:2303.08302*, 2023.

[ZGA+21] Lucia Zheng, Neel Guha, Brandon R Anderson, Peter Henderson, and Daniel E Ho. When does pretraining help? assessing self-supervised learning for law and the casehold dataset of 53,000+ legal holdings. In *Proceedings of the eighteenth international conference on artificial intelligence and law*, pages 159–168, 2021.

[ZSH+22] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.